

KITABU QUEST

S5

COMPUTER SCIENCE



Cover scene

students using computers and code safely with a mountain gorilla mascot

● Clear lessons

● Worked examples

● Fun activities

● Real-life problems

GRADE

5

Title Page

KITABU QUEST S5 Computer Science

Rwanda REB-CBC learner book

Mascot: mountain gorilla

This KITABU QUEST book turns the curriculum into short lessons, worked examples, guided activities, practice, review, and home links for Rwanda learners.

How To Use This Book

This book helps S5 learners study Computer Science one teachable topic at a time.

1. Start with the inquiry question and say what you already know.
2. Read the Learn section slowly and mark new words.
3. Try the worked example before reading the full solution.
4. Complete the activity and practice tasks.
5. Correct your answer using the success criteria.
6. Ask for support when your answer is still unclear.

Subject method: Practise safe computing, algorithms, data, systems thinking, and clear step-by-step explanations.

Learning Skills In This Book

Each topic builds knowledge, skill, values, communication, collaboration, critical thinking, creativity, digital awareness, and self-confidence.

Study actively: speak answers aloud, draw when useful, show your working, cite evidence, use local examples, and correct mistakes after feedback.

A strong answer is specific. It names the idea, gives evidence or method, applies it to the task, and checks whether it answers the question.

Table Of Contents

Use this table to move through the book one topic at a time. Each topic has a lesson opener, clear teaching, a model or example, guided work, practice, and checks for understanding.

1. Fundamental of laptop and portable devices
2. Complex data structure in algorithm
3. Database design
4. Pointers and Structure in C++
5. Object Oriented Programming in C++
6. Variables, Operators, Expressions and Control structures
7. OOP and Java
8. IO and Java
9. Process Management and scheduling algorithm

As you finish each topic, return to the success criteria and correct one answer before moving on.

Fundamental of laptop and portable devices: Lesson Opener

Learning focus: Fundamental of laptop and portable devices

Country link: a safe Rwanda observation, model, data table, or investigation for Fundamental of laptop and portable devices.

By the end of this topic, you should be able to:

- identify, use and maintain safely laptops and other portable devices

Inquiry questions:

- How can fundamental of laptop and portable devices help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Fundamental of laptop and portable devices: Learn

Fundamental of laptop and portable devices is learned by turning the topic into a clear, safe, evidence-based science task.

Core idea:

- Name the science idea before starting: structure, process, material, force, energy, organism, environment, or system.
- Use a diagram, data table, labelled model, safe observation, or teacher-approved investigation.
- Record what is seen or measured, not what is guessed.
- Explain the result using the vocabulary from this topic.
- Finish with one conclusion and one question for further study.

Key words:

- Fundamental of laptop and portable devices
- Fundamental
- laptop
- portable
- devices
- concept
- observation
- evidence

Fundamental of laptop and portable devices: Worked Example

Investigation model for Fundamental of laptop and portable devices: Turn the topic into a safe, clear classroom investigation or model.

1. Question: write one question that can be answered by observing, comparing, classifying, measuring, modelling, or reading data.
2. Materials: list safe classroom materials, diagrams, samples, cards, tables, or teacher-approved apparatus.
3. Method: do one step at a time and keep one condition fair where comparison is needed.
4. Safety: name any heat, glass, sharp tool, chemical, hygiene, electrical, or body-safety rule before starting.
5. Record: use a labelled table, diagram, or sentence notes.
6. Conclusion: state what the evidence shows and one limit of the investigation.

Answer check: a strong response names the concept, uses correct vocabulary, records evidence, and does not claim more than the evidence supports.

Fundamental of laptop and portable devices: Vocabulary And Study Skill

Use these words and study moves before you practise Fundamental of laptop and portable devices. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Fundamental of laptop and portable devices
- Fundamental
- laptop
- portable
- devices
- concept
- observation
- evidence

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Fundamental of laptop and portable devices without a clear example, method, or evidence.

Fundamental of laptop and portable devices: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Fundamental of laptop and portable devices.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Fundamental of laptop and portable devices: Practice And Correction

Practice for Fundamental of laptop and portable devices:

Main outcome: identify, use and maintain safely laptops and other portable devices.

1. Write one safe investigation question.
2. Name materials and observations needed.
3. Use the evidence to write a conclusion.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Fundamental of laptop and portable devices: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Fundamental of laptop and portable devices.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Fundamental of laptop and portable devices: Outcome Check

Outcome check for Fundamental of laptop and portable devices: Identify, use and maintain safely laptops and other portable devices.

1. State a safe investigation question.
2. Name the evidence needed.
3. Write a conclusion from the evidence.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Complex data structure in algorithm: Lesson Opener

Learning focus: Complex data structure in algorithm

Country link: a safe Rwanda observation, model, data table, or investigation for Complex data structure in algorithm.

By the end of this topic, you should be able to:

- utilize complex data structure in algorithm

Inquiry questions:

- How can complex data structure in algorithm help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Complex data structure in algorithm: Learn

Complex data structure in algorithm is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- Complex data structure in algorithm
- Complex
- data
- structure
- algorithm
- concept
- observation
- evidence

Complex data structure in algorithm: Worked Example

Worked example for Complex data structure in algorithm: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

Complex data structure in algorithm: Vocabulary And Study Skill

Use these words and study moves before you practise Complex data structure in algorithm. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Complex data structure in algorithm
- Complex
- data
- structure
- algorithm
- concept
- observation
- evidence

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Complex data structure in algorithm without a clear example, method, or evidence.

Complex data structure in algorithm: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Complex data structure in algorithm.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Complex data structure in algorithm: Practice And Correction

Practice for Complex data structure in algorithm:

Main outcome: utilize complex data structure in algorithm.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Complex data structure in algorithm: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Complex data structure in algorithm.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Complex data structure in algorithm: Outcome Check

Outcome check for Complex data structure in algorithm: Utilize complex data structure in algorithm.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Database design: Lesson Opener

Learning focus: Database design

Country link: a safe Rwanda observation, model, data table, or investigation for Database design.

By the end of this topic, you should be able to:

- define conceptual, logical and physical levels of a database with database creation and manipulation

Inquiry questions:

- How can database design help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Database design: Learn

Database design is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- Database design
- Database
- design
- concept
- observation
- evidence
- safety
- conclusion

Database design: Worked Example

Worked example for Database design: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

Database design: Vocabulary And Study Skill

Use these words and study moves before you practise Database design. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Database design
- Database
- design
- concept
- observation
- evidence
- safety
- conclusion

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Database design without a clear example, method, or evidence.

Database design: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Database design.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Database design: Practice And Correction

Practice for Database design:

Main outcome: define conceptual, logical and physical levels of a database with database creation and manipulation.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Database design: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Database design.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Database design: Outcome Check

Outcome check for Database design: Define conceptual, logical and physical levels of a database with database creation and manipulation.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Pointers and Structure in C++: Lesson Opener

Learning focus: Pointers and Structure in C++

Country link: a safe Rwanda observation, model, data table, or investigation for Pointers and Structure in C++.

By the end of this topic, you should be able to:

- apply pointers for dynamic memory allocation and structure as user defined data type in C ++ program

Inquiry questions:

- How can pointers and structure in c++ help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Pointers and Structure in C++: Learn

Pointers and Structure in C++ is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- Pointers and Structure in C++
- Pointers
- Structure
- concept
- observation
- evidence
- safety
- conclusion

Pointers and Structure in C++: Worked Example

Worked example for Pointers and Structure in C++: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

Pointers and Structure in C++: Vocabulary And Study Skill

Use these words and study moves before you practise Pointers and Structure in C++. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Pointers and Structure in C++
- Pointers
- Structure
- concept
- observation
- evidence
- safety
- conclusion

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Pointers and Structure in C++ without a clear example, method, or evidence.

Pointers and Structure in C++: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Pointers and Structure in C++.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Pointers and Structure in C++: Practice And Correction

Practice for Pointers and Structure in C++:

Main outcome: apply pointers for dynamic memory allocation and structure as user defined data type in C ++ program.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Pointers and Structure in C++: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Pointers and Structure in C++.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Pointers and Structure in C++: Outcome Check

Outcome check for Pointers and Structure in C++: Apply pointers for dynamic memory allocation and structure as user defined data type in C ++ program.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Object Oriented Programming in C++: Lesson Opener

Learning focus: Object Oriented Programming in C++

Country link: a safe Rwanda observation, model, data table, or investigation for Object Oriented Programming in C++.

By the end of this topic, you should be able to:

- explain common concepts of Object Oriented Programming(OOP) and implement them in C++

Inquiry questions:

- How can object oriented programming in c++ help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Object Oriented Programming in C++: Learn

Object Oriented Programming in C++ is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- Object Oriented Programming in C++
- Object
- Oriented
- Programming
- concept
- observation
- evidence
- safety

Object Oriented Programming in C++: Worked Example

Worked example for Object Oriented Programming in C++: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

Object Oriented Programming in C++: Vocabulary And Study Skill

Use these words and study moves before you practise Object Oriented Programming in C++. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Object Oriented Programming in C++
- Object
- Oriented
- Programming
- concept
- observation
- evidence
- safety

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Object Oriented Programming in C++ without a clear example, method, or evidence.

Object Oriented Programming in C++: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Object Oriented Programming in C++.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Object Oriented Programming in C++: Practice And Correction

Practice for Object Oriented Programming in C++:

Main outcome: explain common concepts of Object Oriented Programming(OOP) and implement them in C++.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Object Oriented Programming in C++: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Object Oriented Programming in C++.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Object Oriented Programming in C++: Outcome Check

Outcome check for Object Oriented Programming in C++: Explain common concepts of Object Oriented Programming(OOP) and implement them in C++.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Variables, Operators, Expressions and Control structures: Lesson Opener

Learning focus: Variables, Operators, Expressions and Control structures

Country link: a safe Rwanda observation, model, data table, or investigation for Variables, Operators, Expressions and Control structures.

By the end of this topic, you should be able to:

- use variables, operators, expressions and control structure in a Visual Basic program

Inquiry questions:

- How can variables, operators, expressions and control structures help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Variables, Operators, Expressions and Control structures: Learn

Variables, Operators, Expressions and Control structures is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- Variables, Operators, Expressions and Control structures
- Variables
- Operators
- Expressions
- Control
- structures
- concept
- observation

Variables, Operators, Expressions and Control structures: Worked Example

Worked example for Variables, Operators, Expressions and Control structures: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

Variables, Operators, Expressions and Control structures: Vocabulary And Study Skill

Use these words and study moves before you practise Variables, Operators, Expressions and Control structures. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Variables, Operators, Expressions and Control structures
- Variables
- Operators
- Expressions
- Control
- structures
- concept
- observation

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Variables, Operators, Expressions and Control structures without a clear example, method, or evidence.

Variables, Operators, Expressions and Control structures: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Variables, Operators, Expressions and Control structures.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Variables, Operators, Expressions and Control structures: Practice And Correction

Practice for Variables, Operators, Expressions and Control structures:

Main outcome: use variables, operators, expressions and control structure in a Visual Basic program.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Variables, Operators, Expressions and Control structures: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Variables, Operators, Expressions and Control structures.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Variables, Operators, Expressions and Control structures: Outcome Check

Outcome check for Variables, Operators, Expressions and Control structures: Use variables, operators, expressions and control structure in a Visual Basic program.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

OOP and Java: Lesson Opener

Learning focus: OOP and Java

Country link: a safe Rwanda observation, model, data table, or investigation for OOP and Java.

By the end of this topic, you should be able to:

- use objects to manipulate data in Java program

Inquiry questions:

- How can oop and java help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

OOP and Java: Learn

OOP and Java is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- OOP and Java
- Java
- concept
- observation
- evidence
- safety
- conclusion

OOP and Java: Worked Example

Worked example for OOP and Java: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

OOP and Java: Vocabulary And Study Skill

Use these words and study moves before you practise OOP and Java. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- OOP and Java
- Java
- concept
- observation
- evidence
- safety
- conclusion

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about OOP and Java without a clear example, method, or evidence.

OOP and Java: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for OOP and Java.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

OOP and Java: Practice And Correction

Practice for OOP and Java:

Main outcome: use objects to manipulate data in Java program.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

OOP and Java: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from OOP and Java.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

OOP and Java: Outcome Check

Outcome check for OOP and Java: Use objects to manipulate data in Java program.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

IO and Java: Lesson Opener

Learning focus: IO and Java

Country link: a safe Rwanda observation, model, data table, or investigation for IO and Java.

By the end of this topic, you should be able to:

- use streams, readers and writers in Java

Inquiry questions:

- How can io and java help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

IO and Java: Learn

IO and Java is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- IO and Java
- Java
- concept
- observation
- evidence
- safety
- conclusion

IO and Java: Worked Example

Worked example for IO and Java: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

IO and Java: Vocabulary And Study Skill

Use these words and study moves before you practise IO and Java. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- IO and Java
- Java
- concept
- observation
- evidence
- safety
- conclusion

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about IO and Java without a clear example, method, or evidence.

IO and Java: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for IO and Java.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

IO and Java: Practice And Correction

Practice for IO and Java:

Main outcome: use streams, readers and writers in Java.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

IO and Java: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from IO and Java.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

IO and Java: Outcome Check

Outcome check for IO and Java: Use streams, readers and writers in Java.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Process Management and scheduling algorithm: Lesson Opener

Learning focus: Process Management and scheduling algorithm

Country link: a safe Rwanda observation, model, data table, or investigation for Process Management and scheduling algorithm.

By the end of this topic, you should be able to:

- explain how processes are managed by Operating System and to understand process scheduling algorithms

Inquiry questions:

- How can process management and scheduling algorithm help you solve a real problem in Computer Science?

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Process Management and scheduling algorithm: Learn

Process Management and scheduling algorithm is learned by connecting a computing concept to a clear problem, exact steps, and a checked output.

Core idea:

- Name the computing object first: data, algorithm, program, web page, database, network, device, or security control.
- Break the task into steps before using a computer.
- Use correct technical vocabulary such as input, process, output, variable, file, table, tag, command, or protocol where it fits.
- Test the result with a small example and correct one error.
- Save or present the work in a way another learner can inspect.

Key words:

- Process Management and scheduling algorithm
- Process
- Management
- scheduling
- algorithm
- concept
- observation
- evidence

Process Management and scheduling algorithm: Worked Example

Worked example for Process Management and scheduling algorithm: Build a small checked computing output.

1. Define the task in one sentence.
2. List the inputs needed, such as text, numbers, records, tags, commands, or devices.
3. Write the steps before using the computer.
4. Create a small version of the output: a web page, algorithm trace, table, query, program fragment, network diagram, or security checklist.
5. Test with one example and correct one error.

Answer check: the output should match the task, use correct vocabulary, and be easy for another learner to inspect.

Process Management and scheduling algorithm: Vocabulary And Study Skill

Use these words and study moves before you practise Process Management and scheduling algorithm. Good vocabulary helps you read the question, choose the right method, and explain your answer clearly.

Key words:

- Process Management and scheduling algorithm
- Process
- Management
- scheduling
- algorithm
- concept
- observation
- evidence

Study skill:

1. Choose three key words.
2. Say each word in your own words.
3. Give one example from Rwanda.
4. Use each word in a correct sentence, calculation step, diagram label, or oral answer.
5. Ask a partner to check whether your example matches the topic.

Common mistake to avoid: giving a general answer about Process Management and scheduling algorithm without a clear example, method, or evidence.

Process Management and scheduling algorithm: Guided Activity

Work with a partner or small group.

1. Read the inquiry question again.
2. Choose one real situation linked to a safe Rwanda observation, model, data table, or investigation for Process Management and scheduling algorithm.
3. Decide what evidence, method, vocabulary, tool, text, map, diagram, movement, or example is needed.
4. Complete the task together and explain each step aloud.
5. Compare your answer with another group.
6. Improve one part after feedback.

Group role check: one learner reads the task, one records ideas, one checks vocabulary or working, and one reports the answer.

Process Management and scheduling algorithm: Practice And Correction

Practice for Process Management and scheduling algorithm:

Main outcome: explain how processes are managed by Operating System and to understand process scheduling algorithms.

1. Write the purpose of the computing task in one sentence.
2. Create a small algorithm, code fragment, table, diagram, web page structure, database record, or network/security checklist.
3. Test it with one example and write one correction.

Answer check:

A strong science answer names the idea, uses correct units or labels where needed, records evidence, and writes a conclusion that follows from the evidence.

Safety check: use only teacher-approved materials and never test mains electricity, heat, sharp tools, or chemicals without direct supervision.

Process Management and scheduling algorithm: Challenge And Remediation

Use this page after practice.

If you need support:

1. Re-read the success criteria.
2. Copy the worked example steps as headings.
3. Replace the model details with details from Process Management and scheduling algorithm.
4. Ask one clear question about the step that is confusing.

If you are ready for a challenge:

1. Create a new task from school, home, community, environment, technology, sport, art, market, or fieldwork.
2. Solve or answer it using correct vocabulary.
3. Explain why your answer is reasonable.
4. Write one extension question for a classmate.

Check:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Process Management and scheduling algorithm: Outcome Check

Outcome check for Process Management and scheduling algorithm: Explain how processes are managed by Operating System and to understand process scheduling algorithms.

1. State the computing task or problem.
2. Show the algorithm, code fragment, table, web page structure, database object, diagram, or security control.
3. Test with one example and write one correction.

Success criteria:

- evidence is specific
- safety is considered
- conclusion uses the evidence

Fundamental of laptop and portable devices: Review Clinic 1

Use this review clinic to strengthen Computer Science without copying earlier answers.

1. Choose one idea from Fundamental of laptop and portable devices that was difficult.
2. Explain the idea in your own words.
3. Make one new example from Rwanda.
4. Create one question that checks knowledge.
5. Create one question that checks skill or application.
6. Answer both questions and correct your work.

Challenge: Turn the idea into a drawing, table, map, dialogue, model, demonstration, or worked solution.

Complex data structure in algorithm: Review Clinic 2

Use this review clinic to strengthen Computer Science without copying earlier answers.

1. Choose one idea from Complex data structure in algorithm that was difficult.
2. Explain the idea in your own words.
3. Make one new example from Rwanda.
4. Create one question that checks knowledge.
5. Create one question that checks skill or application.
6. Answer both questions and correct your work.

Challenge: Turn the idea into a drawing, table, map, dialogue, model, demonstration, or worked solution.

Database design: Review Clinic 3

Use this review clinic to strengthen Computer Science without copying earlier answers.

1. Choose one idea from Database design that was difficult.
2. Explain the idea in your own words.
3. Make one new example from Rwanda.
4. Create one question that checks knowledge.
5. Create one question that checks skill or application.
6. Answer both questions and correct your work.

Challenge: Turn the idea into a drawing, table, map, dialogue, model, demonstration, or worked solution.

Pointers and Structure in C++: Review Clinic 4

Use this review clinic to strengthen Computer Science without copying earlier answers.

1. Choose one idea from Pointers and Structure in C++ that was difficult.
2. Explain the idea in your own words.
3. Make one new example from Rwanda.
4. Create one question that checks knowledge.
5. Create one question that checks skill or application.
6. Answer both questions and correct your work.

Challenge: Turn the idea into a drawing, table, map, dialogue, model, demonstration, or worked solution.

Glossary

Use this glossary to revise important words in Computer Science.

- Fundamental of laptop and portable devices: Explain this term using an example from the book, your school, or your community.
- Complex data structure in algorithm: Explain this term using an example from the book, your school, or your community.
- Database design: Explain this term using an example from the book, your school, or your community.
- Pointers and Structure in C++: Explain this term using an example from the book, your school, or your community.
- Object Oriented Programming in C++: Explain this term using an example from the book, your school, or your community.
- Variables, Operators, Expressions and Control structures: Explain this term using an example from the book, your school, or your community.
- OOP and Java: Explain this term using an example from the book, your school, or your community.
- IO and Java: Explain this term using an example from the book, your school, or your community.
- Process Management and scheduling algorithm: Explain this term using an example from the book, your school, or your community.

Add five more words from your lessons and write your own meanings.

Answer And Teacher Notes

A strong Computer Science answer states the idea, uses correct scientific vocabulary, records observations or data, includes units or labels where needed, and writes a conclusion that follows from evidence. Safety rules always come first. If evidence is missing, repeat the observation safely or explain what evidence would be needed.

Final Project

Create a final project for Computer Science that shows what you can now do independently.

Your project must include:

1. A clear title.
2. A real-life problem, question, text, performance, investigation, design, or worked task.
3. Evidence from at least three lessons in this book.
4. A drawing, table, map, chart, model, dialogue, paragraph, performance plan, practical product, or worked solution.
5. A correction note showing one improvement after feedback.
6. A short reflection explaining what you learned, what was difficult, and what you will practise next.

Presentation check: keep the work neat, label important parts, use correct vocabulary, and be ready to explain your choices to a classmate or teacher.